Notes on Markov Decision Processes

Luis Mendez

Discount Factor

The discount factor γ is a parameter in reinforcement learning that determines the importance of future rewards compared to immediate rewards. It is a value between 0 and 1, where: - A value of 0 means that the agent only cares about immediate rewards and ignores future rewards We want to maximize the total discounted reward, which is given by the formula:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Where G_t is the total discounted reward at time step t, and R_{t+k+1} is the reward received at time step $t+k+1$.

Policy $\pi(s)$

A policy is a strategy that the agent follows to determine which action to take in each state. It is a mapping from states to actions, and is denoted by $\pi(a|s)$, which represents the probability of taking action a in state s.

stochastic policy: $\pi(a|s)$ is a probability distribution over actions given a state. It allows for randomness in action selection, which can be useful for exploration. deterministic policy: $\pi(s)$ is a function that maps each state to a specific action. It does not allow for randomness, and the agent will always take the same action in a given state.

We want to maximize the sum of discounted rewards, which can be expressed as:

$$J(\pi) = \sum_s d^\pi(s)\ sum_a \pi(a|s) Q^\pi(s,a)$$

Where $J(\pi)$ is the expected return of the policy π, $d^\pi(s)$ is the stationary distribution of states under policy π, and $Q^\pi(s,a)$ is the action-value function, which represents the expected return of taking action a in state s and following policy π thereafter.

We must find the optimal policy π^* that maximizes $J(\pi)$, which can be expressed as:

$$\pi^* = \arg\max_\pi J(\pi)$$

State Value

The state value function, denoted as $V^\pi(s)$, represents the expected return (total discounted reward) when starting from state s and following policy π thereafter. It can be defined as:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

Where $V^\pi(s)$ is the state value function for state s under policy π, and the expectation is taken over the distribution of rewards and states that result from following policy π starting from state s.

State-Action Value

The state-action value function, denoted as $Q^\pi(s,a)$, represents the expected return (total discounted reward) when starting from state s, taking action a, and following policy π thereafter. It can be defined as:

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Where $Q^\pi(s,a)$ is the state-action value function for state s and action a under policy π, and the expectation is taken over the distribution of rewards and states that result from following policy π starting from state s and taking action a.

Bellman Equations

The Bellman equations are fundamental recursive relationships that relate the value of a state or state-action pair to the values of its successor states or state-action pairs. They are used to compute the value functions and to derive optimal policies. The Bellman equation for the state value function is given by:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right]$$

Where $P(s'|s,a)$ is the transition probability of moving from state s to state s' after taking action a, and $R(s,a,s')$ is the reward received for taking action a in state s and transitioning to state s'. The Bellman equation for the state-action value function is given by:

$$Q^\pi(s,a) = \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right]$$

Where the terms are defined as in the previous equation.

Decision Process

A Markov Decision Process (MDP) is a mathematical framework for modeling decision-making problems where outcomes are partly random and partly under the control of a decision-maker. An MDP is defined by the following components: - A set of states S: This represents all possible states that the system can be in

The optimal value of a state is the expected return when starting from that state and following the optimal policy thereafter. It can be defined as:

$$V^*(s) = \max_\pi V^\pi(s)$$

Where $V^*(s)$ is the optimal state value function for state s, and the maximum is taken over all possible policies π.

The optimal action-value function is the expected return when starting from a state, taking an action, and following the optimal policy thereafter. It can be defined as:

$$Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

Where $Q^*(s,a)$ is the optimal state-action value function for state s and action a, and the maximum is taken over all possible policies π.

Optimal Policy

The optimal policy π^* is the policy that maximizes the expected return for all states. It can be defined as:

$$\pi^*(s) = \arg\max_{a} Q^*(s,a)$$

Where $\pi^*(s)$ is the optimal policy for state s, and the maximum is taken over all possible actions a.

I. Dynamic Programming

Dynamic programming is a method for solving complex problems by breaking them down into simpler subproblems. In the context of MDPs, dynamic programming algorithms are used to compute the value functions and optimal policies. The two main dynamic programming algorithms for MDPs are: - Value Iteration: This algorithm iteratively updates the value function until it converges to the optimal value function. The update rule is given by:

$$V_{k+1}(s) = \max_{a} \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V_k(s') \right]$$

Where $V_k(s)$ is the value function at iteration k, and the maximum is taken over all possible actions a. - Policy Iteration: This algorithm alternates between policy evaluation and policy improvement steps. In the policy evaluation step, the value function for the current policy is computed using the Bellman equation. In the policy improvement step, the policy is updated to be greedy with respect to the current value function. The update rule for the policy improvement step is given by:

$$\pi_{k+1}(s) = \arg\max_{a} \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^{\pi_k}(s') \right]$$

Where $\pi_k(s)$ is the policy at iteration k, and the maximum is taken over all possible actions a.